Tricky Javascript questions

JavaScript Output Questions

These questions are commonly asked in interviews to test **JavaScript fundamentals, hoisting, event loop, closures, and object behavior**.

---

1. Object.freeze vs Object.seal

Code

JAVASCRIPT Copy

```
let person1 = {
  name: "ravi",
  age: 21
};

let person2 = {
  name: "ram",
  age: 43
};

person1 = Object.freeze(person1);
person2 = Object.seal(person2);

person1.age = 31;
person2.age = 53;
```

```
console.log(person1.location, person1.age);
console.log(person2.location, person2.age);
```

Output

JAVASCRIPT Copy

```
undefined 21
undefined 53
```

Explanation

Object.freeze

- Prevents modifying existing properties
- Prevents adding new properties

So `person1` stays unchanged.

Object.seal

- Allows modification of existing properties
- Prevents adding new properties

So `person2.age` changes but `location` is not added.

---

2. Functions are Objects

Code

JAVASCRIPT Copy

```
}

bark.animal = "dog";
```

Explanation

Functions in JavaScript are **objects**, so you can attach properties to them.

JAVASCRIPT Copy

```
console.log(bark.animal);
```

Output:

JAVASCRIPT Copy

```
dog
```

---

3. Strict Mode and Global Variables

Code

JAVASCRIPT Copy

```
function getAge() {
  "use strict";
  age = 21;
  console.log(age);
}

getAge();
```

JAVASCRIPT Copy

```
ReferenceError
```

Explanation

In strict mode, assigning a value to an undeclared variable throws an error.

Without strict mode, `age` would become a global variable.

---

4. typeof with Arrow Functions

Code

JAVASCRIPT Copy

```
function sayHi() {
  return (() => 0)();
}

console.log(typeof sayHi());
```

Output

JAVASCRIPT Copy

```
number
```

Explanation

5. Sparse Arrays

Code

JAVASCRIPT Copy

```javascript
const numbers = [1, 2, 3];
numbers[10] = 11;

console.log(numbers);
```

Output

JAVASCRIPT Copy

```
[1, 2, 3, empty × 7, 11]
```

Explanation

JavaScript creates **empty slots** between index `3` and `9`.

This is called a **sparse array**.

6. Variable Leakage

Code

JAVASCRIPT Copy

```
  var a = (b = 10);
}

display();

console.log(typeof b === "undefined");
console.log(typeof a === "undefined");
}
```

Output

JAVASCRIPT Copy

```
false
true
```

Explanation

JAVASCRIPT Copy

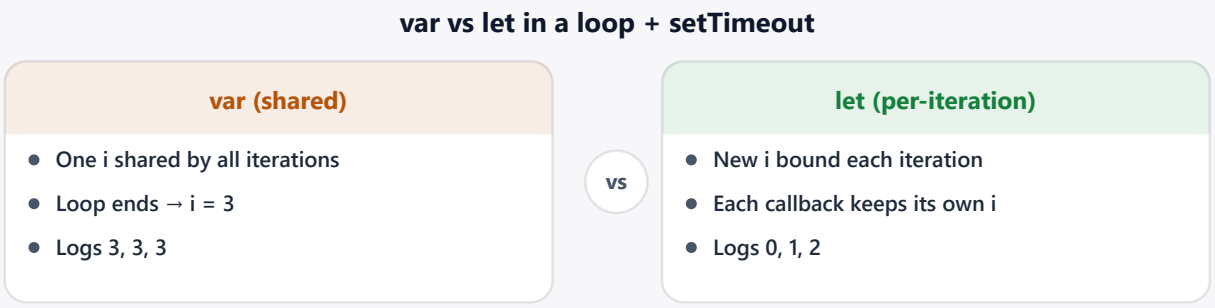
```
var a = (b = 10)
```

means

JAVASCRIPT Copy

```
b = 10
var a = b
```

- `b` becomes a **global variable**
- `a` remains local to the function



let is block-scoped, so every loop turn captures a fresh binding — the classic closure fix.

Code

JAVASCRIPT Copy

```javascript
for (var i = 0; i < 3; i++) {
  setTimeout(() => console.log(i), i * 10);
}
```

Output

JAVASCRIPT Copy

```
3
3
3
```

Explanation

`var` is function scoped.

By the time callbacks run, `i` becomes `3`.

---

8. Variable Redeclaration

JAVASCRIPT Copy

```javascript
var num = 8;
var num = 10;

console.log(num);
```

Output

JAVASCRIPT Copy

```
10
```

Explanation

`var` allows **re-declaration**, so the second assignment overwrites the first.

---

9. Event Loop Order

Code

JAVASCRIPT Copy

```javascript
const foo = () => console.log("First");
const bar = () => setTimeout(() => console.log("Second"));
const baz = () => console.log("Third");

bar();
foo();
baz();
```

Output

JAVASCRIPT Copy

```
First
Third
Second
```

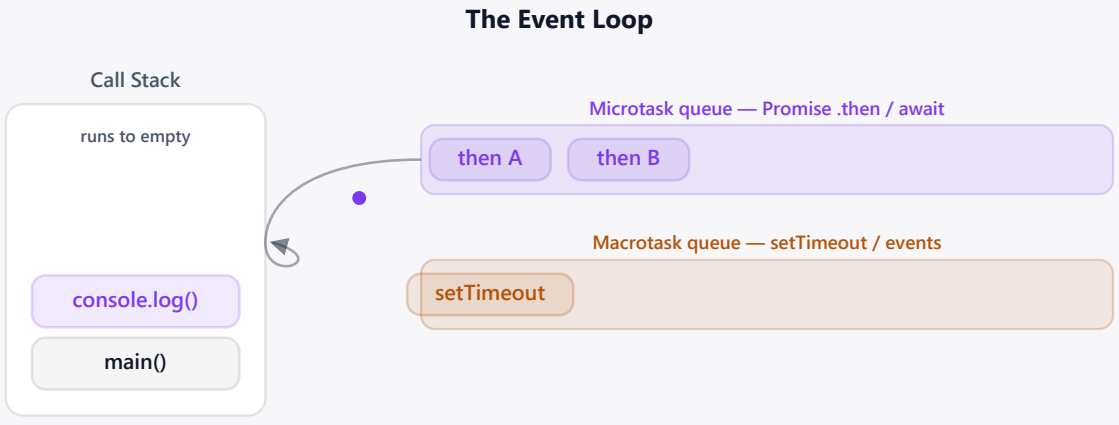
Explanation

Execution order:

1. `bar()` schedules setTimeout
2. `foo()` runs
3. `baz()` runs
4. setTimeout runs later

---

10. Microtasks vs Macrotasks



When the stack is empty: drain ALL microtasks, then take ONE macrotask — and repeat.

This ordering is why a Promise callback always runs before a setTimeout(..., 0) scheduled at the same time.

Code

```javascript
console.log("one");

setTimeout(function () {
  console.log("two");
}, 0);

Promise.resolve().then(function () {
  console.log("three");
});

console.log("four");
```

Output

JAVASCRIPT Copy

```
one
four
three
two
```

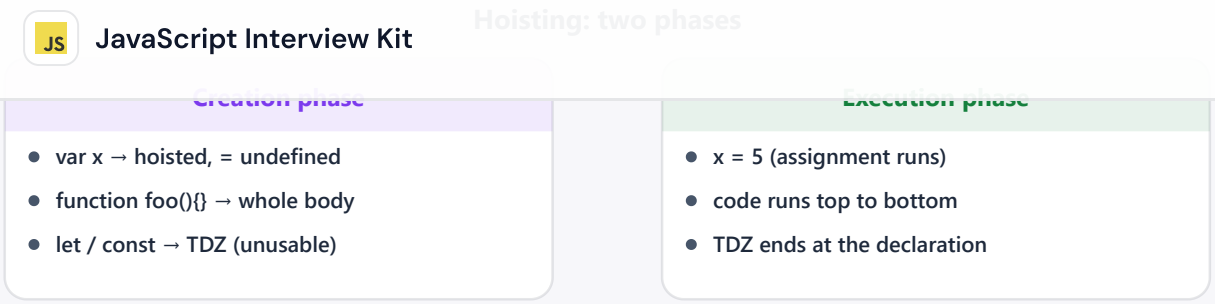
Explanation

Execution order:

1. synchronous code
2. microtasks (Promises)
3. macrotasks (setTimeout)

---

11. Hoisting Example



Declarations are set up first; assignments happen when the line actually runs. That gap explains most hoisting puzzles.

Code

JAVASCRIPT Copy

```javascript
var a = 1;

if (true) {
  function a() {}
  var a = 10;
}

console.log(a);
```

Output

JAVASCRIPT Copy

```
10
```

---

12. Function vs Variable Hoisting

Code

JAVASCRIPT Copy

```javascript
function foo() {
  console.log("1");
}

var foo = 9;

console.log(typeof foo);
```

Output

JAVASCRIPT Copy

```
function
number
```

Explanation

Function declarations are hoisted before variables.

---

13. Variable Hoisting in Functions

Code

JAVASCRIPT Copy

```javascript
var name = "outer";

function foo() {
  console.log(name);
  var name = "inner";
}
```

Output

JAVASCRIPT Copy

```
undefined
```

Explanation

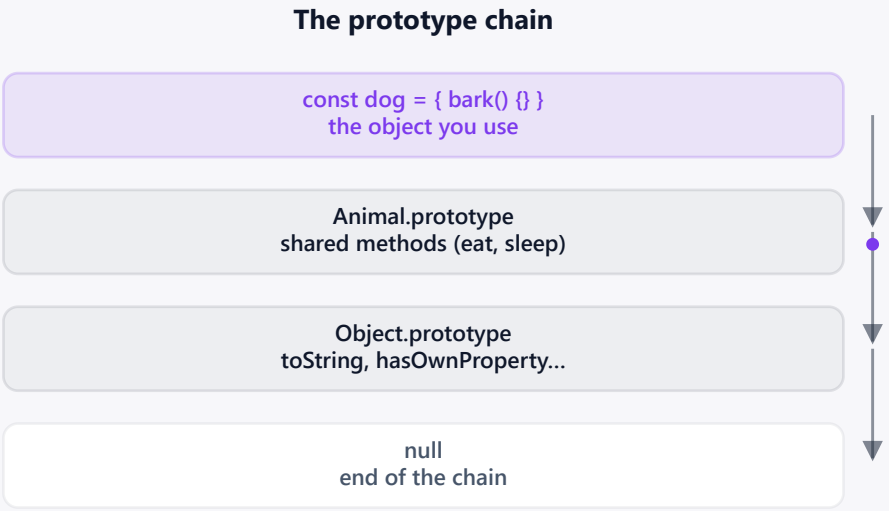
Inside the function:

JAVASCRIPT Copy

```
var name;
console.log(name);
name = "inner";
```

---

14. Prototype Inheritance



Miss a property on the object? JS walks UP the chain until it finds it or hits null. That lookup is prototypal inheritance.

JAVASCRIPT Copy

```
const a = { foo: 123 };
const b = Object.create(a);

console.log(b.foo);

b.foo = 444;
console.log(b.foo);

delete b.foo;
console.log(b.foo);
```

Output

JAVASCRIPT Copy

```
123
444
123
```

Explanation

`b` inherits `foo` from `a`.

Deleting `b.foo` exposes the prototype property again.

---

15. arguments Object Trick

Code

JAVASCRIPT Copy

```javascript
function callback() {
  console.log(this.length);
}

const object = {
  length: 5,
  method() {
    arguments[0]();
  }
};

object.method(callback, 1, 2);
```

Output

JAVASCRIPT Copy

```
3
```

---

16. arguments vs normal call

Code

JAVASCRIPT Copy

```javascript
var length = 4;

function callback() {
  console.log(this.length);
}

const object = {
  length: 5,
  method(callback) {
```

```
};

object.method(callback, 1, 2);
```

Output

JAVASCRIPT Copy

```
4
```

---

17. Function Hoisting with Duplicate Names

Code

JAVASCRIPT Copy

```
function x() {
  a();
  function a() {
    console.log("m");
  }

  a();

  function a() {
    console.log("n");
  }

  a();
}

x();
```

JAVASCRIPT Copy

```
n
n
n
```

Explanation

Function declarations are hoisted.

The **last declaration overrides previous ones**.

---

18. Async Await Execution

Code

JAVASCRIPT Copy

```javascript
async function data() {
  console.log("3");

  await new Promise((res) => setTimeout(res, 1000));

  console.log("4");
}

console.log("1");
data();
console.log("2");
```

Output

JAVASCRIPT Copy

```
2
4
```

19. Promise + setTimeout

Code

JAVASCRIPT Copy

```javascript
console.log("1");

setTimeout(() => {
  console.log("3");
});

Promise.resolve("4").then(() => setTimeout(() => console.log("4")));

Promise.resolve("5").then((data) => console.log(data));

console.log("2");
```

Output

JAVASCRIPT Copy

```
1
2
5
3
4
```

Code

JAVASCRIPT Copy

```javascript
function Person(firstName, lastName) {
  this.firstName = firstName;
  this.lastName = lastName;
}

const lydia = new Person("Lydia", "Hallie");
const sarah = Person("Sarah", "Smith");

console.log(lydia);
console.log(sarah);
```

Output

JAVASCRIPT Copy

```javascript
Person { firstName: "Lydia", lastName: "Hallie" }
undefined
```

---

21. Generator Functions

Code

JAVASCRIPT Copy

```javascript
function* generatorFunction() {
  yield 1;
  yield 2;
  return 3;
```

```
}
const generator = generatorFunction();

console.log(generator.next());
console.log(generator.next());
console.log(generator.next());
```

Output

JAVASCRIPT Copy

```
{ value: 1, done: false }
{ value: 2, done: false }
{ value: 3, done: true }
```

---

22. Prototype Chain Lookup

Question

What will be printed?

JAVASCRIPT Copy

```
const animal = {
  sound: "generic"
};

const dog = Object.create(animal);

dog.sound = "bark";

delete dog.sound;

console.log(dog.sound);
```

Answer

JAVASCRIPT Copy

```
generic
```

Explanation

Steps:

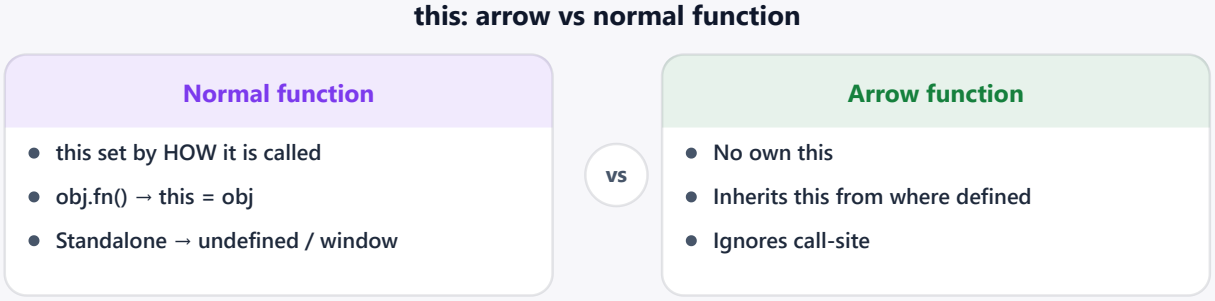
1. `dog` inherits from `animal`
2. `dog.sound = "bark"` creates its own property
3. `delete dog.sound` removes the property from `dog`
4. JavaScript looks up the **prototype chain**

So it finds:

JAVASCRIPT Copy

```
animal.sound → "generic"
```

23. Arrow Function and `this`



Question

JAVASCRIPT Copy

```javascript
const obj = {
  name: "JS",
  show: () => {
    console.log(this.name);
  }
};

obj.show();
```

Answer

JAVASCRIPT Copy

```javascript
undefined
```

Explanation

Arrow functions **do not have their own** `this`.

They inherit `this` from the **lexical scope**.

In this case:

JAVASCRIPT Copy

```javascript
this → global object
```

Global object does not have `name`.

So output:

JAVASCRIPT Copy

```
undefined
```

24. setTimeout and Closure with let

Question

JAVASCRIPT Copy

```
for (let i = 0; i < 3; i++) {
  setTimeout(() => console.log(i), 10);
}
```

Answer

JAVASCRIPT Copy

```
0
1
2
```

Explanation

`let` is **block scoped**.

Each loop iteration creates a **new binding of** `i`.

So callbacks capture:

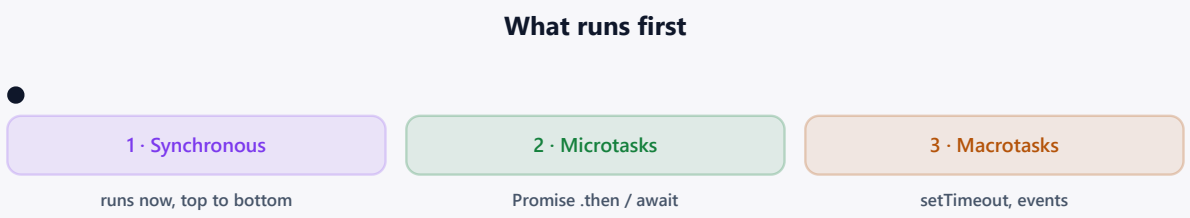
JAVASCRIPT Copy

```
0
1
2
```

instead of `3`.

---

25. Promise Execution Order



All sync code, THEN every queued microtask, THEN one macrotask. Memorize this order and output questions become easy.

Question

JAVASCRIPT Copy

```
console.log("Start");

Promise.resolve().then(() => {
  console.log("Promise 1");
});

Promise.resolve().then(() => {
  console.log("Promise 2");
});

console.log("End");
```

Answer

JAVASCRIPT Copy

```
Start
End
Promise 1
Promise 2
```

Explanation

Execution order:

1. Synchronous code

JAVASCRIPT Copy

```
Start
End
```

2. Microtasks (Promises)

JAVASCRIPT Copy

```
Promise 1
Promise 2
```

Promises run **after the call stack is empty**.

---

26. Object Reference

JAVASCRIPT Copy

```
const obj1 = { a: 1 };
const obj2 = obj1;

obj2.a = 5;

console.log(obj1.a);
```

Answer

JAVASCRIPT Copy

```
5
```

Explanation

Objects are assigned **by reference**, not by value.

JAVASCRIPT Copy

```
obj1 → memory location
obj2 → same memory location
```

Changing `obj2` also changes `obj1`.

27. Array Reference Behavior

JAVASCRIPT Copy

```
const arr = [1,2,3];

const copy = arr;

copy.push(4);

console.log(arr);
```

Answer

JAVASCRIPT Copy

```
[1,2,3,4]
```

Explanation

Arrays are also **reference types**.

JAVASCRIPT Copy

```
arr → reference
copy → same reference
```

So modifying one affects the other.

28. Spread Operator Copy

JAVASCRIPT Copy

```
const arr = [1,2,3];

const copy = [ ... arr];

copy.push(4);

console.log(arr);
console.log(copy);
```

Answer

JAVASCRIPT Copy

```
[1,2,3]
[1,2,3,4]
```

Explanation

Spread operator creates a **shallow copy**.

So:

JAVASCRIPT Copy

```
arr and copy are different arrays
```

Changing one does not affect the other.

Question

JAVASCRIPT Copy

```
console.log([] == []);
console.log([] === []);
```

Answer

JAVASCRIPT Copy

```
false
false
```

Explanation

Arrays are **objects**, so equality compares **references**.

Each array is stored at a different memory location.

JAVASCRIPT Copy

```
[] !== []
```

30. NaN Comparison

Question

```
console.log(NaN === NaN);
console.log(Object.is(NaN, NaN));
```

Answer

JAVASCRIPT Copy

```
false
true
```

Explanation

`NaN` is the **only value in JavaScript that is not equal to itself**.

JAVASCRIPT Copy

```
NaN === NaN → false
```

But:

JAVASCRIPT Copy

```
Object.is(NaN, NaN) → true
```

because `Object.is()` checks values more precisely.

QUICK CHECK — no marks, just for you.

JAVASCRIPT Copy

```
let config = Object.freeze({ mode: "dark", size: 10 });
let draft = Object.seal({ mode: "light", size: 20 });

config.size = 99;
draft.size = 99;

config.owner = "a";
draft.owner = "b";

console.log(config.size, config.owner);
console.log(draft.size, draft.owner);
```

- ◯ TypeError
- ◯ 10 undefined
99 b
- ◯ 99 undefined
99 undefined
- ◯ 10 undefined
99 undefined

Check answer

2. What is the exact output order?

JAVASCRIPT Copy

```
console.log("one");

setTimeout(function () {
  console.log("two");
}, 0);

Promise.resolve().then(function () {
```

```
console.log("four");
```

- one
  two
  three
  four
- one
  three
  four
  two
- one
  four
  two
  three
- one
  four
  three
  two

Check answer

3. What does this snippet log?

JAVASCRIPT Copy

```
console.log(typeof foo);

function foo() {
  console.log("1");
}

var foo = 9;

console.log(typeof foo);
```

- `function`
  `function`
- `number`
  `number`
- `function`
  `number`

Check answer

Next
Polyfill coding questions

LEARN MORE

ARTICLES & DOCS

- JavaScript output questions (repo)
  github.com
- The Modern JavaScript Tutorial
  javascript.info

VIDEOS



Solving output-based JS questions
YouTube



Top 20 output-based questions
YouTube



JS output tricky questions

JavaScript Interview Kit

Hand-picked references for this lesson.